

Universitat Politècnica de Catalunya
ETSETB / FIB

Aprendizaje no supervisado y detección de anomalías

Documento de descripción de los problemas de la exposición

Computació d'Altes Prestacions i Intel · ligència Artificial (CPIA)

Equipo:
Matías, Martí, Uriel y Arnau Montull
Grado en Ingeniería Física



27 de octubre de 2026

Índice

1. Equipo y reparto de problemas **2**

2. Fundamentos comunes **2**

3. Problema de Arnau Montull: detección de explosiones nucleares con DBSCAN **3**
3.1. Planteamiento y originalidad . . . 3
3.2. Datos . . . 3
3.3. Fundamentación teórica . . . 4
3.4. Método: cómo se detecta . . . 4
3.5. Elección de parámetros . . . 5
3.6. Validación de ε y estabilidad (`nukeval.py`) . . . 6
3.7. Resultados . . . 7
3.8. Conclusiones . . . 8
3.9. Cómo reproducirlo . . . 8
3.10. Código fuente . . . 8

4. Problema de Matías: agrupación de sismos y detección de anomalías **12**

5. Problema de Martí: transcriptómica (RNA-seq) **13**

6. Problema de Uriel **14**

7. Problema de Uriel **15**

8. Conclusiones del equipo **15**

1. Equipo y reparto de problemas

Exposición del tema «Aprendizaje no supervisado y detección de anomalías». El objetivo es entender cómo encontrar estructura y patrones en datos sin etiquetar con clustering (K-Means, DBSCAN, jerárquico), mezclas gaussianas e Isolation Forest, y cómo evaluar un clustering sin etiquetas (silhouette y método del codo).

Integrante	Problema	Algoritmo principal	Fichero
Arnau Montull	Detección de explosiones nucleares entre sismos (catálogo USGS)	DBSCAN (haversine)	`nukeDBSCAN_arnau.py`
Matías	Agrupación de sismos del último mes y detección de anomalías	K-Means ($K = 3$) y k-vecinos más próximos	`presentation_matias.py`, `validacio.py`
Martí	Transcriptómica (RNA-seq): módulos de genes co-expresados y muestras anómalas	Clustering jerárquico, K-Means, DBSCAN, Isolation Forest	**[PENDIENTE: fichero .py (aún sin código)]**
Uriel	**[PENDIENTE: título del problema]**	**[PENDIENTE: algoritmo]**	**[PENDIENTE: fichero]**

Tabla 1: Reparto de problemas. Cada fichero se llama `<nemotécnico>_<estudiante>.py`.

2. Fundamentos comunes

Resumen de los conceptos que usan los problemas. La explicación completa de K-Means (en catalán) está en `docs/teoria/fonaments_de_clustering_i_kmeans.tex`.

Aprendizaje no supervisado y clustering. Se busca estructura en datos sin etiquetas. Cada algoritmo necesita una medida de similitud (distancia euclídea, haversine, correlación…) y la elección condiciona el resultado.

K-Means. Divide los datos en K grupos minimizando la suma de distancias cuadráticas de cada punto al centroide de su grupo, $J = \sum_{k=1}^{K} \sum_{x \in C_k} \|x - \mu_k\|^2$. Se resuelve con el algoritmo de Lloyd (asignar cada punto al centroide más cercano y recalcular centroides, hasta converger), inicializado con K-means++. Es un clustering *duro*, supone grupos compactos y de tamaño parecido, y hay que fijar K.

Cómo elegir K sin etiquetas. Método del codo (la inercia deja de bajar de forma apreciable), coeficiente de silueta, $s = (b - a)/\max(a, b)$, con a la distancia media a los puntos de su grupo y b la distancia media al grupo vecino más próximo, e índice de Davies–Bouldin.

DBSCAN. Agrupa por densidad: un punto es *central* si tiene al menos $v_{\text{mín}}$ puntos (él incluido) a distancia $\leq \varepsilon$; los grupos son las componentes conexas de puntos centrales y los puntos que no alcanza ninguno quedan como ruido (-1). No necesita fijar el número de grupos y encuentra formas arbitrarias.

Clustering jerárquico, mezclas gaussianas e Isolation Forest. El jerárquico construye un dendrograma según el tipo de enlace (simple, medio, Ward) y se corta a una altura. Las mezclas gaussianas (`GaussianMixture`) dan una asignación blanda por probabilidad. Isolation Forest puntúa como anómalos los puntos que se aíslan con pocos cortes aleatorios. **[PENDIENTE: el equipo revisa y amplía este resumen con lo que se exponga.]**

Comentarios / observaciones:

3. Problema de Arnau Montull: detección de explosiones nucleares con DBSCAN

3.1. Planteamiento y originalidad

Los sismógrafos registran por igual terremotos y explosiones nucleares. Preguntamos si un clustering que no mira la etiqueta puede señalar las explosiones solo con la posición y la profundidad del evento. El agrupamiento (DBSCAN) es no supervisado y la ventana de profundidad se fija con un criterio físico, sin optimizarla contra la etiqueta; esta solo se usa para evaluar (ver las limitaciones). La idea física es doble: las pruebas nucleares se repiten en los mismos polígonos de ensayo (Nevada, Semipalátinsk, Novaya Zemlya, Lop Nor, Mururoa…), por lo que forman agrupaciones espaciales muy compactas, y ocurren casi en la superficie, mientras que los terremotos tectónicos tienen profundidad mediana de unos 22 km.

Es un problema de aplicación física que no figura entre las sugerencias del enunciado. Los datos son públicos y el trabajo es de detección y verificación, no una guía para ocultar pruebas.

3.2. Datos

Catálogo del USGS (*ComCat*) con todos los eventos de magnitud $M \geq 4{,}5$ entre 1951 y 2025 (`datos/sismos_usgs_M45_1951_2025.csv`), descargado con `nukedatos.py`. No se filtra por tipo de evento. Tras quitar los eventos sin latitud, longitud, profundidad o magnitud quedan 298 127: 614 etiquetados por el USGS como `nuclear explosion`, 297 413 `earthquake` y 100 de otros tipos (71 erupciones volcánicas, 16 `explosion`, 6 colapsos de mina, 3 estallidos de roca, 2 deslizamientos, 1 voladura de cantera y 1 `other event`). La columna `tipo` *solo se usa para evaluar*, nunca para entrenar.

`nukedatos.py` pide a la API del USGS un año cada vez (cada consulta devuelve como máximo 20 000 eventos), reintenta si falla la red y guarda el resultado cambiando el nombre de la columna `type` por `tipo`. El código completo está en el Anexo.

```
13 def descargar_year(year):
14     # La API limita cada consulta a 20 000 eventos, asi que se pide un año cada
       vez
```

```
    consulta = f'{URL}&starttime={year}-01-01&endtime={year␣+␣1}-01-01'

    for intento in range(4):
        try:
            return pd.read_csv(consulta)
        except Exception:
            print(f'␣␣{year}:␣reintento')

    raise RuntimeError(f'No␣se␣pudo␣descargar␣{year}')
```

Listing 1: `nukedatos.py`: descarga de un año del catálogo, con reintentos.

`nukeDBSCAN_arnau.py` carga el CSV y se queda con los eventos que tienen posición, profundidad y magnitud.

```
# --- Datos ---
df = pd.read_csv(ARCHIVO)
df = df.dropna(subset=['latitude', 'longitude', 'depth', 'mag'])
df = df[df['mag'] >= 4.5]
df = df.reset_index(drop=True)
```

Listing 2: `nukeDBSCAN_arnau.py`: carga y limpieza de los datos.

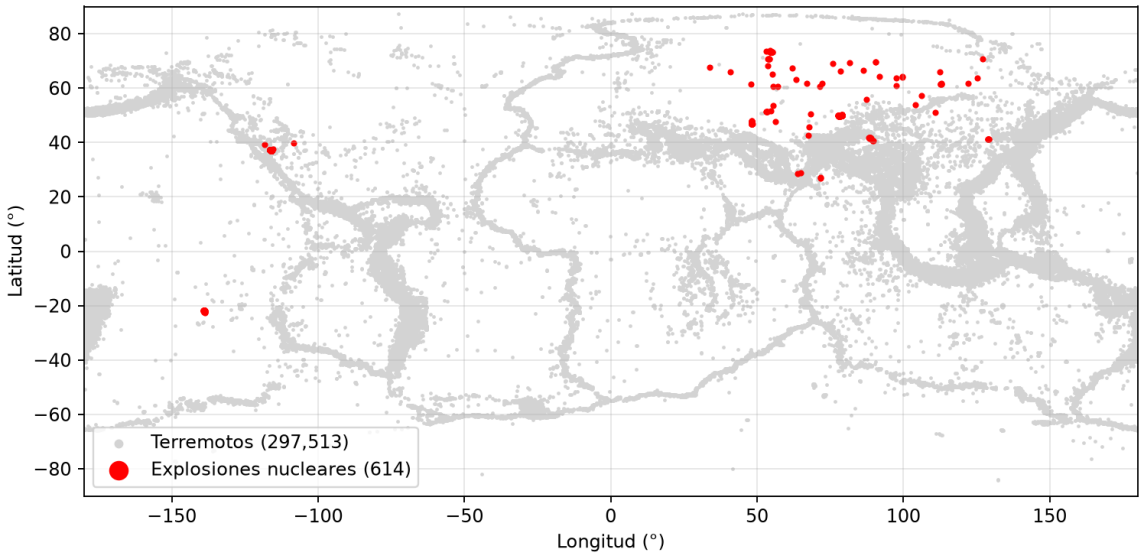


Figura 1: Catálogo completo: las explosiones (rojo) se concentran en unos pocos polígonos de ensayo.

3.3. Fundamentación teórica

DBSCAN [1] define los grupos por densidad. Dado ε y $v_{\text{mín}}$, un punto es central si su vecindad de radio ε contiene al menos $v_{\text{mín}}$ puntos; dos puntos centrales a distancia $\leq \varepsilon$ pertenecen al mismo grupo (alcanzabilidad por densidad) y los puntos sin vecindad suficiente que no alcanza ningún punto central son ruido. Con $v_{\text{mín}} = 1$ todo punto es central y los grupos son las componentes conexas del grafo que une los sismos a menos de ε; es equivalente al clustering jerárquico de enlace simple cortado a la altura ε.

Los epicentros están sobre la esfera, así que la distancia correcta es la ortodrómica. La fórmula de *haversine* [3],

$$d = 2R \arcsin \sqrt{\sin^2 \tfrac{\Delta\varphi}{2} + \cos\varphi_1 \cos\varphi_2 \sin^2 \tfrac{\Delta\lambda}{2}}, \qquad R = 6371 \text{ km},$$

es numéricamente estable para distancias pequeñas y está implementada en `sklearn.cluster.DBSCAN` [2] (`metric='haversine'`, con ε expresado en radianes, $\varepsilon_{\text{rad}} = \varepsilon_{\text{km}}/R$). Aquí el clustering solo agrupa por posición; el criterio físico (profundidad) es el que etiqueta cada grupo.

Como las clases están muy desbalanceadas (614 explosiones frente a casi 300 000 terremotos), la exactitud no sirve: un detector que nunca marcase nada tendría un 99,8 % de aciertos. Se usan la precisión $P = \mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$, el recall $R = \mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$ y su media armónica, $F_1 = 2PR/(P+R) = 2\,\mathrm{TP}/(2\,\mathrm{TP} + \mathrm{FP} + \mathrm{FN})$.

3.4. Método: cómo se detecta

1. **Agrupar por posición.** DBSCAN con distancia *haversine* (distancia sobre la esfera; las coordenadas se pasan a radianes y el radio ε se divide por el radio terrestre, 6371 km). Dos sismos pertenecen al mismo cluster si hay una cadena de sismos a menos de ε km uno del siguiente.
2. **Mirar la profundidad de cada cluster.** Se calcula la *mediana* de profundidad de cada cluster (la mediana no se deja engañar por unos pocos valores atípicos).
3. **Marcar como sospechosos** los clusters cuya mediana está en $[d_{\mathrm{mín}}, d_{\mathrm{máx}})$: casi en la superficie. Todos los sismos de un cluster sospechoso se marcan como *detectados*.
4. **Evaluar contra la etiqueta.** Acierto: detectado y explosión. Falso positivo (FP): detectado pero no explosión. Falso negativo (FN): explosión no detectada.

El paso 1 es la función `agrupar`: pasa las coordenadas a radianes, divide ε entre el radio terrestre y llama a DBSCAN con la métrica *haversine*.

```
# --- Agrupar los sismos cercanos con DBSCAN ---
def agrupar(df, eps_km, vmin):
    radio_tierra_km = 6371
    coordenadas = np.radians(df[['latitude', 'longitude']])
    eps_rad = eps_km / radio_tierra_km

    dbscan = DBSCAN(eps=eps_rad, min_samples=vmin, metric='haversine')
    cluster = dbscan.fit_predict(coordenadas)
    return pd.Series(cluster, index=df.index)
```

Listing 3: `agrupar`: DBSCAN con distancia haversine.

Los pasos 2 y 3 son `nukeDBSCAN`: la mediana de profundidad de cada cluster y la selección de los que caen en la ventana.

```
# --- Marcar los sismos sospechosos de ser explosiones ---
def nukeDBSCAN(df, eps_km, vmin, dmin, dmax):
    cluster = agrupar(df, eps_km, vmin)

    # Profundidad mediana de cada cluster (el ruido, -1, no cuenta)
    sin_ruido = cluster != -1
    mediana = df['depth'].groupby(cluster[sin_ruido]).median()

    # Sospechoso: cluster casi en la superficie, con la mediana entre dmin y dmax
    es_superficial = (mediana >= dmin) & (mediana < dmax)
    clusters_sospechosos = mediana[es_superficial].index

    return cluster.isin(clusters_sospechosos)
```

Listing 4: `nukeDBSCAN`: clusters sospechosos por la mediana de profundidad.

El paso 4 es `errores`, que cuenta FP y FN con la etiqueta `tipo`.

```
# --- Falsos positivos y falsos negativos ---
def errores(df, detectado):
    es_explosion = df['tipo'] == 'nuclear␣explosion'

    # Falso positivo: detectado, pero no es una explosion
    falsos_positivos = (detectado & ~es_explosion).sum()
```

```
    # Falso negativo: es una explosion, pero no se detecta
    falsos_negativos = (~detectado & es_explosion).sum()
    return falsos_positivos, falsos_negativos
```

Listing 5: `errores`: falsos positivos y falsos negativos.

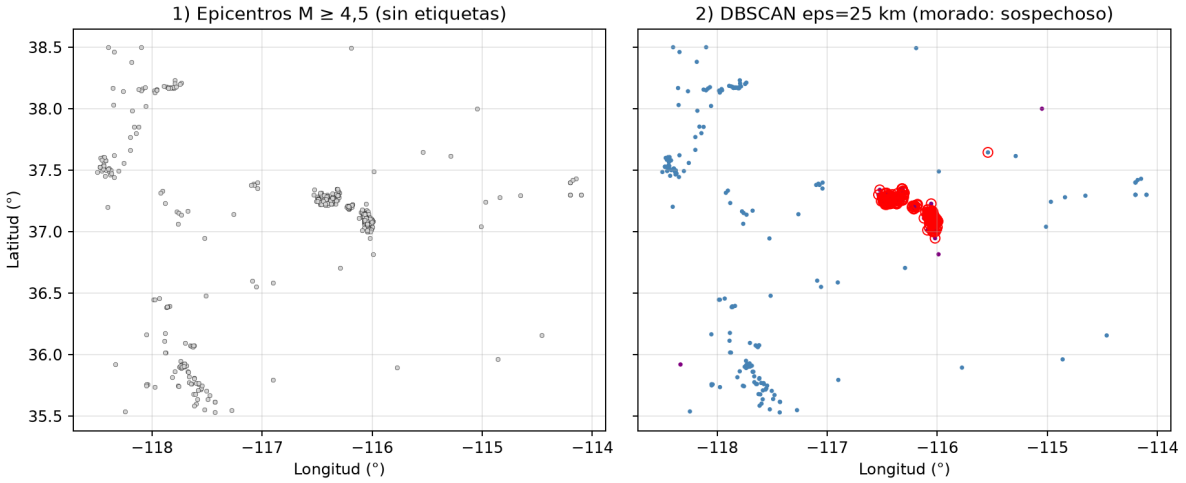


Figura 2: Zoom en el polígono de Nevada. Izquierda: epicentros sin etiquetar. Derecha: clusters de DBSCAN; en morado, los clusters sospechosos por profundidad; círculos rojos: explosiones reales (etiqueta USGS).

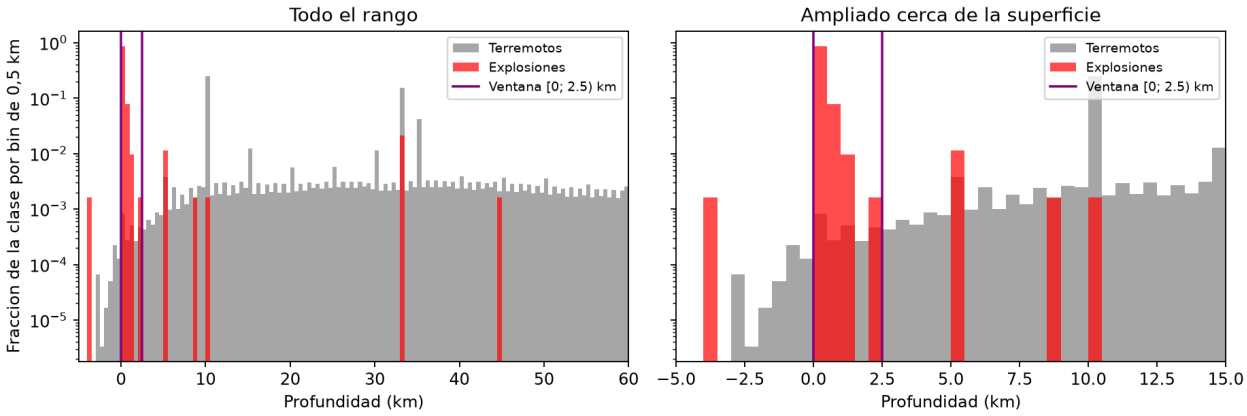


Figura 3: Distribución de profundidades: fracción de cada clase en bins de 0,5 km (escala logarítmica y mismo eje vertical), completa y ampliada cerca de la superficie. Las líneas moradas delimitan la ventana $[d_{\text{mín}}, d_{\text{máx}})$, que se aplica a la *mediana* de cada cluster y no a cada sismo.

3.5. Elección de parámetros

Parámetros fijos: $\varepsilon = 25$ km, $v_{\text{mín}} = 1$, $d_{\text{mín}} = 0$ y $d_{\text{máx}} = 2{,}5$ km. Están en un único diccionario al principio de `nukeDBSCAN_arnau.py`, que `nukeval.py` importa.

```
# Parametros fijos
#   eps_km: radio de DBSCAN. vmin: sismos minimos por cluster.
#   dmin, dmax: una explosion es casi superficial, asi que un cluster es
    sospechoso
#   si la mediana de su profundidad esta entre 0 y 2.5 km.
parametros = dict(eps_km=25, vmin=1, dmin=0, dmax=2.5)
```

Listing 6: `nukeDBSCAN_arnau.py`: parámetros fijos.

- **Ventana de profundidad** [0; 2,5) **km:** criterio físico. Una explosión nuclear enterrada está a menos de unos pocos km de la superficie; los terremotos tectónicos están, en su mayoría, mucho más profundos.
- $\varepsilon = 25$ **km:** del orden del tamaño de un polígono de ensayo. Es el valor con menos falsos positivos de los probados (5 a 100 km, ver la validación): con menos radio los terremotos superficiales quedan en grupos sueltos y con más radio los grupos de explosiones se funden con terremotos más profundos. Aquí sí se mira la etiqueta.
- $v_{\text{mín}} = 1$**:** con $v_{\text{mín}} = 3$ empeora (ver las limitaciones).

El script `nuke_optimizar.py` (búsqueda de la ventana y de ε por F_1) se probó, pero no se usa para los parámetros finales: su óptimo, [0; 0,1) km, queda en el borde de su rejilla y aprovecha que el USGS registra con profundidad exactamente 0,0 km el 85 % de las explosiones (521 de 614) y solo 154 de 297 513 eventos que no son explosiones nucleares. Eso mide cómo cataloga el USGS y no la física. La tabla compara ventanas con $\varepsilon = 25$ km.

Ventana (km)	TP	FP	FN	F_1
[0; 0,1) (la que elegía el optimizador)	585	58	29	0,931
[0; 2,5) (usada)	586	115	28	0,891
[0; 5)	587	363	27	0,751
[0; 10)	588	2 897	26	0,287

Tabla 2: Efecto de la ventana de profundidad ($\varepsilon = 25$ km, $v_{\text{mín}} = 1$).

3.6. Validación de ε y estabilidad (`nukeval.py`)

Siguiendo el esquema de validación del temario (silueta, codo y estabilidad), `nukeval.py` evalúa ε con $v_{\text{mín}} = 1$ y la ventana [0; 2,5) km. Para cada ε cuenta los clusters, calcula la silueta con distancia haversine sobre una muestra de 2 000 sismos (sin etiquetas) y los FP y FN (con etiquetas).

```
# --- Silueta de unos clusters (sobre una muestra de 2000 sismos) ---
def silueta(df, cluster):
    coordenadas = np.radians(df[['latitude', 'longitude']])
    try:
        return silhouette_score(coordenadas, cluster, metric='haversine',
                                sample_size=2000, random_state=42)
    except ValueError:
        # La muestra no tiene al menos 2 clusters
        return np.nan


# --- Para cada eps: numero de clusters, silueta, FP y FN ---
def evaluar_eps(df, vmin, dmin, dmax):
    print('\n--- ELECCION DE EPS ---')
    print(f"{'eps (km)':<10}{'clusters':<10}{'silueta':<10}{'FP':<8}{'FN':<8}")
    print('-' * 46)

    n_clusters = []
    siluetas = []
    for eps in EPS_PROBADOS:
        cluster = agrupar(df, eps, vmin)
        detectado = nukeDBSCAN(df, eps, vmin, dmin, dmax)
        fp, fn = errores(df, detectado)

        n_clusters.append(cluster.nunique())
        siluetas.append(silueta(df, cluster))
        print(f'{eps:<10}{n_clusters[-1]:<10}{siluetas[-1]:<10.3f}{fp:<8}{fn:<8}'
    )

```

```
46      return n_clusters, siluetas
```

Listing 7: `nukeval.py`: silueta de unos clusters y tabla para cada ε probado.

La estabilidad compara los clusters del catálogo completo con los de cinco muestras aleatorias del 50 % mediante el ARI.

```
59  # --- Estabilidad: ARI entre los clusters del catalogo y los de una muestra del
        50% ---
60  def evaluar_estabilidad(df, eps_km, vmin):
61      print(f'\n--- ESTABILIDAD (eps={eps_km} km, muestra del 50%) ---')
62      cluster_completo = agrupar(df, eps_km, vmin)
63
64      for semilla in [1, 2, 3, 4, 5]:
65          muestra = df.sample(frac=0.5, random_state=semilla)
66          cluster_muestra = agrupar(muestra, eps_km, vmin)
67
68          ari = adjusted_rand_score(cluster_completo[muestra.index],
        cluster_muestra)
69          print(f'Semilla {semilla}: clusters={cluster_muestra.nunique()}, ARI={ari
        :.4f}')
```

Listing 8: `nukeval.py`: estabilidad con el Adjusted Rand Index.

ε (km)	Clusters	Silueta	FP	FN
5	111 117	0,01	500	15
10	44 804	−0,18	251	13
25	11 049	−0,45	115	28
40	5 691	−0,43	123	30
50	4 214	−0,50	483	28
100	1 786	−0,51	151	269

Tabla 3: Validación de ε (extracto; `nukeval.py` imprime 10 valores).

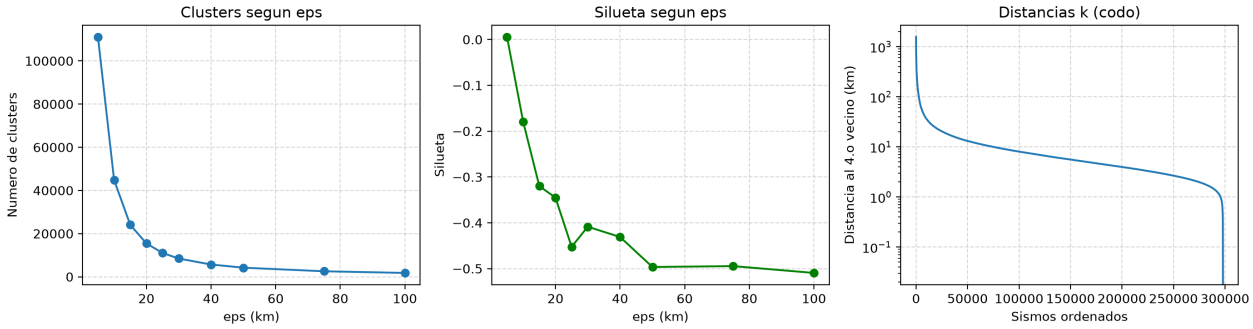


Figura 4: Clusters, silueta y distancia al 4.º vecino (codo clásico de DBSCAN) frente a ε.

- **La silueta no elige ε:** es negativa y decrece al aumentar ε. Con distancia por densidad, DBSCAN une cadenas alargadas de sismos y la silueta (que premia grupos compactos) lo penaliza. Tampoco el gráfico de distancias al 4.º vecino muestra un codo claro entre 20 y 30 km.
- **Lo que sí limita ε es la profundidad:** los FP bajan al crecer ε hasta un mínimo de 115 en 25 km (con ε pequeño muchos terremotos superficiales quedan en grupos sueltos); a partir de 40–50 km los grupos de explosiones se funden con terremotos más profundos: los FP suben a 483 con 50 km y los FN pasan de 28 a 261 con 75 km, porque la mediana deja de ser superficial.

- **Estabilidad:** el ARI entre el clustering de todo el catálogo y el de una muestra aleatoria del 50 % (5 semillas, $\varepsilon = 25$ km) vale entre 0,54 y 0,65: moderado, porque al quitar sismos se rompen cadenas de DBSCAN.

3.7. Resultados

Con los parámetros fijos: 586 aciertos, 115 FP y 28 FN, es decir, precisión 83,6 %, recall 95,4 % y $F_1 = 0{,}891$.

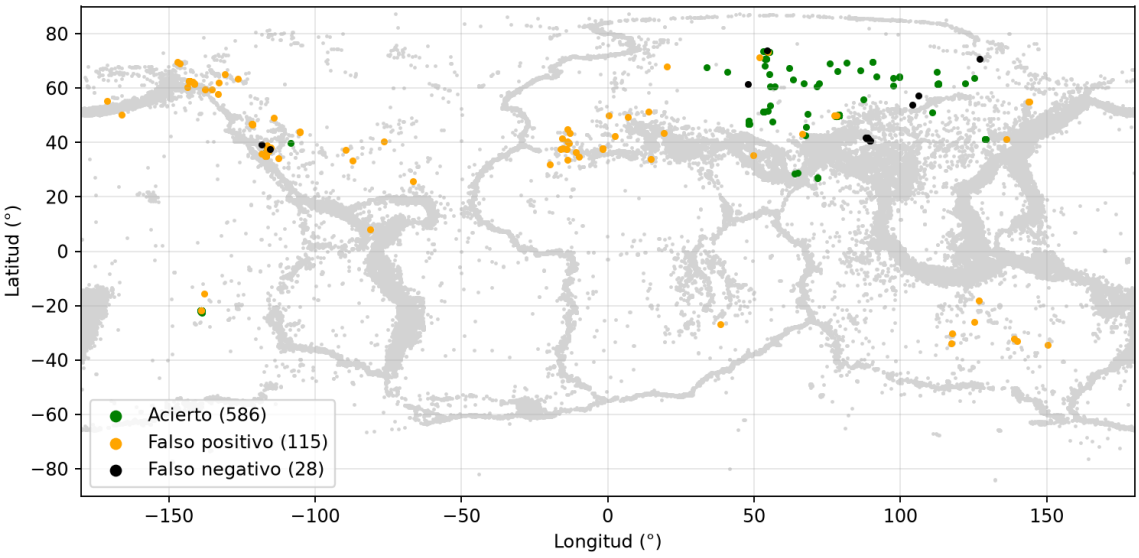


Figura 5: Resultado final: aciertos (verde), falsos positivos (naranja) y falsos negativos (negro).

Limitaciones y honestidad del resultado.

- **Con $v_{\text{mín}} = 1$ no hay ruido:** cada sismo aislado forma su propio cluster. DBSCAN actúa aquí como un agrupador por proximidad encadenada, y la selección real la hace el filtro de profundidad. Con $v_{\text{mín}} = 3$ el resultado empeora ($F_1 = 0{,}899$; FN = 69), porque muchas pruebas aisladas se perderían como «ruido».
- **Enfoque híbrido, no puramente no supervisado:** DBSCAN agrupa sin etiquetas y la ventana de profundidad es un criterio físico previo, pero ε se ha escogido mirando el F_1 contra la etiqueta entre unos pocos valores, porque la silueta y el codo no lo determinan (ver la validación).
- **Generalización:** no hay un subconjunto de prueba separado: se ajusta y se evalúa en el mismo catálogo.
- **Convención del catálogo:** el 85 % de las explosiones figura con profundidad exactamente 0,0 km. Una ventana más estrecha, [0; 0,1), daría $F_1 = 0{,}931$ pero mediría esa convención del USGS y no la física.
- **Falsos positivos:** terremotos muy superficiales (años 80 en Washington, California, Alaska) y otros eventos no sísmicos. De los 115 FP, 24 son de otro tipo: 16 `explosion`, 5 colapsos de mina, 1 voladura de cantera, 1 estallido de roca y 1 deslizamiento; ninguna de las 71 erupciones volcánicas se marca. Si se descartasen los otros 100 eventos no sísmicos (solo terremotos y explosiones nucleares), serían 91 FP y $F_1 = 0{,}908$. Algunas pruebas oficiales de Nevada figuran en el USGS como `earthquake`; ese error es de la etiqueta, no del método.
- **Sesgo de época:** no se usan el año ni el número de estaciones como variable, porque delatan la época del catálogo y no la física.

Comentarios / observaciones:

3.8. Conclusiones

- Un clustering espacial sin etiquetas, combinado con la profundidad mediana de cada grupo, recupera el 95 % de las explosiones del catálogo ($F_1 = 0{,}891$) con 115 falsos positivos entre casi 300 000 eventos.
- La señal útil es doble y física: repetición en los mismos polígonos de ensayo y profundidad casi nula.
- Líneas de mejora: elegir ε en un subconjunto y evaluarlo en otro; añadir la magnitud y la forma de onda como variables; y usar la lista oficial DOE/NV-209 [5] en lugar de la etiqueta USGS para las pruebas de Nevada.

3.9. Cómo reproducirlo

1. Crear el entorno: `python -m venv .venv` e instalar `requirements.txt` (numpy, pandas, matplotlib, scikit-learn, plotly).
2. Opcional: `python problemas/2_Nuclear/nukedatos.py` descarga del USGS el catálogo (`sismos_usgs_M45_` a `datos/`; si el archivo ya existe, no hace nada.
3. `python problemas/2_Nuclear/nukeDBSCAN_arnau.py` (se puede lanzar desde cualquier carpeta). Imprime FP y FN del run con los parámetros fijos y escribe `globo.html` (globo interactivo con 4 vistas, funciona sin internet).
4. `python problemas/2_Nuclear/nuke_optimizar.py` reproduce la búsqueda por F_1 que se descartó (opcional; unos minutos).
5. `python problemas/2_Nuclear/nukeval.py` reproduce la validación de ε y la estabilidad (unos 4 min) y guarda `nukeval_eps.png` y `nukeval_mapa.png`.
6. Las figuras de este documento: `python docs/figuras_nuclear.py`.

4. Problema de Matías: agrupación de sismos y detección de anomalías

Material en `problemas/3_Sismes_Matias/` (README, código, datos e informe propio en catalán, `informe_sismes.tex`). Lo que sigue resume su `README.md` y su código; *no he ejecutado su código ni he comprobado sus resultados.*

Objetivo. Segmentar los sismos globales del último mes con K-Means ($K = 3$) según magnitud, profundidad, latitud y longitud, para identificar patrones de subducción y actividad en los bordes de placa, y detectar eventos excepcionales (alta magnitud o profundidad inusual) como anomalías.

Datos. `earthquakes.csv`: 10 174 eventos del catálogo USGS con cabecera estándar (`time, latitude, longitude, depth, mag, ...`); el primero es del 15/10/2026.

Método.

1. Carga, reordenación de columnas y filtrado de registros incompletos o inválidos.
2. Normalización z-score de las cuatro variables (media 0, desviación 1).
3. K-Means con $K = 3$.
4. Anomalías: distancia euclídea media a los $k = 5$ vecinos más próximos en el espacio normalizado; son anómalos los puntos más aislados.

Validación. Método del codo (WCSS), coeficiente de silueta y *Adjusted Rand Index* (ARI) entre inicializaciones distintas para medir la estabilidad; `validacio.py` también genera el mapa geográfico.

Reproducir. `pip install -r requirements.txt` (pandas, numpy, scikit-learn, matplotlib), después `python presentation_matias.py` y `python validacio.py`. Hay versiones `*_PANDAS.py` de ambos scripts. Nota: el README nombra `problema_matias.py`, que no existe en la carpeta, y los ficheros no siguen el formato `<nemotécnico>_<estudiante>.py` que pide el enunciado.

[PENDIENTE: Matías: resultados (valor de K elegido, silueta, ARI, anomalías encontradas) y su conclusión.]

Comentarios / observaciones:

5. Problema de Martí: transcriptómica (RNA-seq)

Material en `problemas/4_RNAseq_Marti/` (documento `EXPO CPIA - Clustering.docx`). Es por ahora un *plan*, sin código.

Objetivo. Identificar módulos de genes co-expresados y detectar muestras anómalas en datos de RNA-seq con aprendizaje no supervisado. La secuenciación mide la expresión de miles de genes a la vez y no hay etiquetas, porque no se sabe de antemano qué genes funcionan juntos: es un problema de alta dimensionalidad.

Datos propuestos. `scanpy.datasets.pbmc3k()` (2 700 células sanguíneas humanas) o el conjunto UCI *Gene Expression Cancer RNA-Seq* de Kaggle (801 muestras de pacientes con distintos tipos de cáncer, en CSV).

Método previsto.

- Preprocesado: normalizar y aplicar logaritmo (los recuentos son asimétricos y dependen de la profundidad de secuenciación).
- Distancia: correlación en vez de euclídea, porque importa el patrón de expresión y no el valor absoluto.
- Clustering jerárquico con dendrograma: enlaces simple, medio y Ward, y cómo se corta el árbol.
- Validación sin etiquetas: silueta y codo para elegir k.
- Comparación con K-Means y DBSCAN.
- Isolation Forest para las muestras problemáticas.
- Conclusiones y limitaciones: maldición de la dimensionalidad, sensibilidad al preprocesado, datos sintéticos frente a reales.

[PENDIENTE: Martí: código `.py`, resultados y figuras.]

Comentarios / observaciones:

6. Problema de Uriel

[PENDIENTE: Uriel: título, datos, método, parámetros, resultados y código.]

Comentarios / observaciones:

7. Problema de Uriel

[PENDIENTE: descripción detallada para reproducir el experimento: datos, método, parámetros, resultados, código.]

Comentarios / observaciones:

8. Conclusiones del equipo

[PENDIENTE: comparativa de los cuatro problemas, qué algoritmo funcionó mejor y por qué.]

Comentarios / observaciones:

Anexo

Anexo: código completo del problema de Arnau

Los tres programas del problema de las explosiones nucleares, tal como se ejecutaron. Cada sección del informe cita solo las líneas que explican en ese punto.

```
"""
Descarga el catalogo del USGS (todos los eventos M>=4.5 de 1951 a 2025, de
    cualquier tipo)
y lo guarda en datos/sismos_usgs_M45_1951_2025.csv.
"""
from pathlib import Path
import pandas as pd

DATOS = Path(__file__).resolve().parent.parent.parent / 'datos'
DESTINO = DATOS / 'sismos_usgs_M45_1951_2025.csv'
URL = 'https://earthquake.usgs.gov/fdsnws/event/1/query?format=csv&orderby=time-
    asc&minmagnitude=4.5'


def descargar_year(year):
    # La API limita cada consulta a 20 000 eventos, asi que se pide un año cada
    vez
    consulta = f'{URL}&starttime={year}-01-01&endtime={year + 1}-01-01'

    for intento in range(4):
        try:
            return pd.read_csv(consulta)
        except Exception:
            print(f'  {year}: reintento')

    raise RuntimeError(f'No se pudo descargar {year}')


if __name__ == '__main__':
    if DESTINO.exists():
        print(f'{DESTINO.name}: ya esta')
    else:
        DATOS.mkdir(exist_ok=True)

        years = []
        for year in range(1951, 2026):
            eventos = descargar_year(year)
            years.append(eventos)
            print(f'{year}: {len(eventos)} eventos')

        df = pd.concat(years, ignore_index=True)
        df = df.rename(columns={'type': 'tipo'})
        df.to_csv(DESTINO, index=False)

        explosiones = (df['tipo'] == 'nuclear explosion').sum()
        print(f'{DESTINO.name}: {len(df)} eventos, {explosiones} explosiones')
```

Listing 9: `problemas/2_Nuclear/nukedatos.py`: descarga del catálogo.

```
from pathlib import Path
import numpy as np
import pandas as pd
import plotly.graph_objects as go
from sklearn.cluster import DBSCAN

# --- Catalogo y parametros ---
# Catalogo USGS 1951-2025, M>=4.5, todos los eventos (se descarga con nukedatos.
    py)
CARPETA = Path(__file__).resolve().parent
ARCHIVO = CARPETA.parent.parent / 'datos' / 'sismos_usgs_M45_1951_2025.csv'
```

```

# Parametros fijos
#   eps_km: radio de DBSCAN. vmin: sismos minimos por cluster.
#   dmin, dmax: una explosion es casi superficial, asi que un cluster es sospechoso
#   si la mediana de su profundidad esta entre 0 y 2.5 km.
parametros = dict(eps_km=25, vmin=1, dmin=0, dmax=2.5)

# --- Datos ---
df = pd.read_csv(ARCHIVO)
df = df.dropna(subset=['latitude', 'longitude', 'depth', 'mag'])
df = df[df['mag'] >= 4.5]
df = df.reset_index(drop=True)


# --- Agrupar los sismos cercanos con DBSCAN ---
def agrupar(df, eps_km, vmin):
    radio_tierra_km = 6371
    coordenadas = np.radians(df[['latitude', 'longitude']])
    eps_rad = eps_km / radio_tierra_km

    dbscan = DBSCAN(eps=eps_rad, min_samples=vmin, metric='haversine')
    cluster = dbscan.fit_predict(coordenadas)
    return pd.Series(cluster, index=df.index)


# --- Marcar los sismos sospechosos de ser explosiones ---
def nukeDBSCAN(df, eps_km, vmin, dmin, dmax):
    cluster = agrupar(df, eps_km, vmin)

    # Profundidad mediana de cada cluster (el ruido, -1, no cuenta)
    sin_ruido = cluster != -1
    mediana = df['depth'].groupby(cluster[sin_ruido]).median()

    # Sospechoso: cluster casi en la superficie, con la mediana entre dmin y dmax
    es_superficial = (mediana >= dmin) & (mediana < dmax)
    clusters_sospechosos = mediana[es_superficial].index

    return cluster.isin(clusters_sospechosos)


# --- Falsos positivos y falsos negativos ---
def errores(df, detectado):
    es_explosion = df['tipo'] == 'nuclear explosion'

    # Falso positivo: detectado, pero no es una explosion
    falsos_positivos = (detectado & ~es_explosion).sum()
    # Falso negativo: es una explosion, pero no se detecta
    falsos_negativos = (~detectado & es_explosion).sum()
    return falsos_positivos, falsos_negativos


if __name__ == '__main__':
    # --- Run con los parametros fijos ---
    detectado = nukeDBSCAN(df, **parametros)
    fp, fn = errores(df, detectado)
    print(f'{parametros}: FP={fp}, FN={fn}')

    # --- Grupos de sismos para el globo ---
    es_explosion = df['tipo'] == 'nuclear explosion'
    terremotos_muestra = df[~es_explosion].sample(4000, random_state=0)
    explosiones = df[es_explosion]
    sospechosos = df[detectado]
    aciertos = df[detectado & es_explosion]
    falsos_positivos = df[detectado & ~es_explosion]
```

```
    falsos_negativos = df[~detectado & es_explosion]

    # Cada vista del globo es una lista de (sismos, nombre, color)
    vistas = {
        'Todos los eventos': [(terremotos_muestra, 'Terremotos (muestra)', '
    lightgray'),
                              (explosiones, 'Explosiones nucleares', 'red')],
        'Sospechosos': [(sospechosos, 'Sospechosos', 'purple')],
        'Reales': [(explosiones, 'Explosiones nucleares', 'red')],
        'Aciertos y errores': [(aciertos, 'Acierto', 'green'),
                               (falsos_positivos, 'Falso positivo', 'orange'),
                               (falsos_negativos, 'Falso negativo', 'gray')],
    }

    # --- Globo (globo.html) ---
    fig = go.Figure()
    for vista, capas in vistas.items():
        for sismos, nombre, color in capas:
            fig.add_trace(go.Scattergeo(
                lat=sismos['latitude'],
                lon=sismos['longitude'],
                mode='markers',
                marker=dict(color=color, size=5),
                name=f'{nombre} ({len(sismos)})',
                hoverinfo='skip',
                meta=vista,
                visible=(vista == 'Todos los eventos'),
            ))

    # Un boton por vista: muestra las capas de esa vista y oculta las demas
    botones = []
    for vista in vistas:
        visibles = [capa.meta == vista for capa in fig.data]
        botones.append(dict(label=vista, method='update', args=[{'visible':
    visibles}]))

    fig.update_layout(updatemenus=[dict(buttons=botones, x=0.01, y=0.99)],
                      legend=dict(x=0.01, y=0.1),
                      margin=dict(l=0, r=0, t=0, b=0))
    fig.update_geos(projection_type='natural earth',
                    showland=True, landcolor='whitesmoke', showcountries=True)

    # Plotly va incluido en el HTML: funciona sin internet
    fig.write_html(CARPETA / 'globo.html', include_plotlyjs=True)
    fig.show()
```

Listing 10: `problemas/2_Nuclear/nukeDBSCAN_arnau.py`: detección y globo interactivo.

```
"""
Validacion de nukeDBSCAN_arnau.py: silueta y distancias k para mirar eps,
estabilidad con el Adjusted Rand Index (ARI) y mapa de aciertos y errores.
"""
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.neighbors import NearestNeighbors

from nukeDBSCAN_arnau import parametros, df, agrupar, nukeDBSCAN, errores

CARPETA = Path(__file__).resolve().parent
RADIO_TIERRA_KM = 6371
EPS_PROBADOS = [5, 10, 15, 20, 25, 30, 40, 50, 75, 100]


```

```
# --- Silueta de unos clusters (sobre una muestra de 2000 sismos) ---
def silueta(df, cluster):
    coordenadas = np.radians(df[['latitude', 'longitude']])
    try:
        return silhouette_score(coordenadas, cluster, metric='haversine',
                                sample_size=2000, random_state=42)
    except ValueError:
        # La muestra no tiene al menos 2 clusters
        return np.nan


# --- Para cada eps: numero de clusters, silueta, FP y FN ---
def evaluar_eps(df, vmin, dmin, dmax):
    print('\n--- ELECCION DE EPS ---')
    print(f"{'eps (km)':<10}{'clusters':<10}{'silueta':<10}{'FP':<8}{'FN':<8}")
    print('-' * 46)

    n_clusters = []
    siluetas = []
    for eps in EPS_PROBADOS:
        cluster = agrupar(df, eps, vmin)
        detectado = nukeDBSCAN(df, eps, vmin, dmin, dmax)
        fp, fn = errores(df, detectado)

        n_clusters.append(cluster.nunique())
        siluetas.append(silueta(df, cluster))
        print(f'{eps:<10}{n_clusters[-1]:<10}{siluetas[-1]:<10.3f}{fp:<8}{fn:<8}'
    )

    return n_clusters, siluetas


# --- Distancia de cada sismo a su 4.o vecino, de mayor a menor (el "codo" de
    DBSCAN) ---
def distancias_k(df, k=4):
    coordenadas = np.radians(df[['latitude', 'longitude']])
    vecinos = NearestNeighbors(n_neighbors=k, metric='haversine').fit(coordenadas
    )
    distancias, _ = vecinos.kneighbors(coordenadas)

    distancia_k = distancias[:, -1] * RADIO_TIERRA_KM
    return np.sort(distancia_k)[::-1]


# --- Estabilidad: ARI entre los clusters del catalogo y los de una muestra del
    50% ---
def evaluar_estabilidad(df, eps_km, vmin):
    print(f'\n--- ESTABILIDAD (eps={eps_km} km, muestra del 50%) ---')
    cluster_completo = agrupar(df, eps_km, vmin)

    for semilla in [1, 2, 3, 4, 5]:
        muestra = df.sample(frac=0.5, random_state=semilla)
        cluster_muestra = agrupar(muestra, eps_km, vmin)

        ari = adjusted_rand_score(cluster_completo[muestra.index],
    cluster_muestra)
        print(f'Semilla {semilla}: clusters={cluster_muestra.nunique()}, ARI={ari
    :.4f}')


# --- Figura 1: clusters, silueta y distancias k ---
def figura_eps(n_clusters, siluetas, distancia_k):
    fig, ejes = plt.subplots(1, 3, figsize=(14, 3.8))

    ejes[0].plot(EPS_PROBADOS, n_clusters, 'o-')
```

```
    ejes[0].set(xlabel='eps (km)', ylabel='Numero de clusters', title='Clusters segun eps')

    ejes[1].plot(EPS_PROBADOS, siluetas, 'o-', color='green')
    ejes[1].set(xlabel='eps (km)', ylabel='Silueta', title='Silueta segun eps')

    ejes[2].plot(distancia_k)
    ejes[2].set(xlabel='Sismos ordenados', ylabel='Distancia al 4.o vecino (km)',
                yscale='log', title='Distancias k (codo)')

    for eje in ejes:
        eje.grid(True, linestyle='--', alpha=0.5)

    fig.tight_layout()
    fig.savefig(CARPETA / 'nukeval_eps.png', dpi=200)


# --- Figura 2: mapa de aciertos y errores ---
def figura_mapa(df, detectado):
    es_explosion = df['tipo'] == 'nuclear explosion'
    otros = df[~detectado & ~es_explosion]

    fig, eje = plt.subplots(figsize=(10, 5))
    eje.scatter(otros['longitude'], otros['latitude'], s=1, c='lightgray')

    grupos = [(detectado & es_explosion, 'green', 'Acierto'),
              (detectado & ~es_explosion, 'orange', 'Falso positivo'),
              (~detectado & es_explosion, 'black', 'Falso negativo')]
    for mascara, color, nombre in grupos:
        sismos = df[mascara]
        eje.scatter(sismos['longitude'], sismos['latitude'], s=8, c=color,
                    label=f'{nombre} ({len(sismos)})')

    eje.set(xlabel='Longitud (°)', ylabel='Latitud (°)',
            title='Aciertos y errores con los parametros fijos')
    eje.legend(loc='lower left', markerscale=2)
    eje.grid(True, linestyle='--', alpha=0.5)

    fig.tight_layout()
    fig.savefig(CARPETA / 'nukeval_mapa.png', dpi=200)


if __name__ == '__main__':
    print(f'Catalogo: {len(df)} eventos, parametros {parametros}')

    n_clusters, siluetas = evaluar_eps(df, parametros['vmin'], parametros['dmin'], parametros['dmax'])
    evaluar_estabilidad(df, parametros['eps_km'], parametros['vmin'])

    detectado = nukeDBSCAN(df, **parametros)
    fp, fn = errores(df, detectado)
    print(f'\nRun con los parametros fijos: FP={fp}, FN={fn}')

    figura_eps(n_clusters, siluetas, distancias_k(df))
    figura_mapa(df, detectado)
    print(f'Figuras guardadas en: {CARPETA}')
```

Listing 11: `problemas/2_Nuclear/nukeval.py`: validación de ε y estabilidad.

Referencias

[1] Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proc. 2nd Int. Conf. on Knowledge Discovery and*

Data Mining (KDD-96), 226–231.

[2] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[3] Sinnott, R. W. (1984). Virtues of the haversine. *Sky & Telescope*, 68(2), 159.

[4] U.S. Geological Survey. *ANSS Comprehensive Earthquake Catalog (ComCat)*. https://earthquake.usgs.gov/data/comcat/

[5] U.S. Department of Energy, Nevada Operations Office (2015). *United States Nuclear Tests, July 1945 through September 1992*, DOE/NV-209 Rev. 16. https://www.osti.gov/biblio/1351809